

Низькорівневе програмування та програмування мікроконтролерів

З використанням мови програмування **Assembler**
I/O, Interruption

класрум **im3ozqq4**

Input/Output

Ресар

Усі приклади, які ми розглядали до цього моменту, працювали виключно з пам'яттю та регістрами процесора. Проте основна мета програмування полягає не лише у внутрішніх обчисленнях, а у створенні реальних «ефектів». Саме завдяки їм результати роботи програми стають корисними: ми можемо прочитати вхідні дані, зберегти їх у пам'ять чи на носій, передати через порт або відобразити на екрані.

I/O: вивід у консоль

Ми вже стикалися з введенням-виведенням у найпростішій формі — друкували текст у консоль. Як це відбувалося? У більшості випадків ми використовували системні виклики (наприклад, через **int 80h** в Linux або DOS-функції), які дозволяли передати керування від програми до операційної системи. Операційна система вже знала, як взаємодіяти з пристроями — клавіатурою та екраном. Таким чином, наша програма опосередковано «просила» ОС вивести потрібні символи на екран.

INT 80h: двері до ядра ОС

Інструкція **INT 80h** — це виклик програмного переривання в Linux для доступу до системних функцій ядра. Коли програма виконує `int 0x80`, процесор переключає керування на обробник, визначений операційною системою. Перед цим у регістрах (наприклад, EAX, EBX, ECX) програміст повинен розмістити номер функції та її параметри. Ядро читає ці значення, виконує потрібну дію (наприклад, запис у файл чи вивід у консоль) і повертає результат назад у регістр. Таким чином, виклик виглядає як проста інструкція, але фактично активує складний механізм ОС.

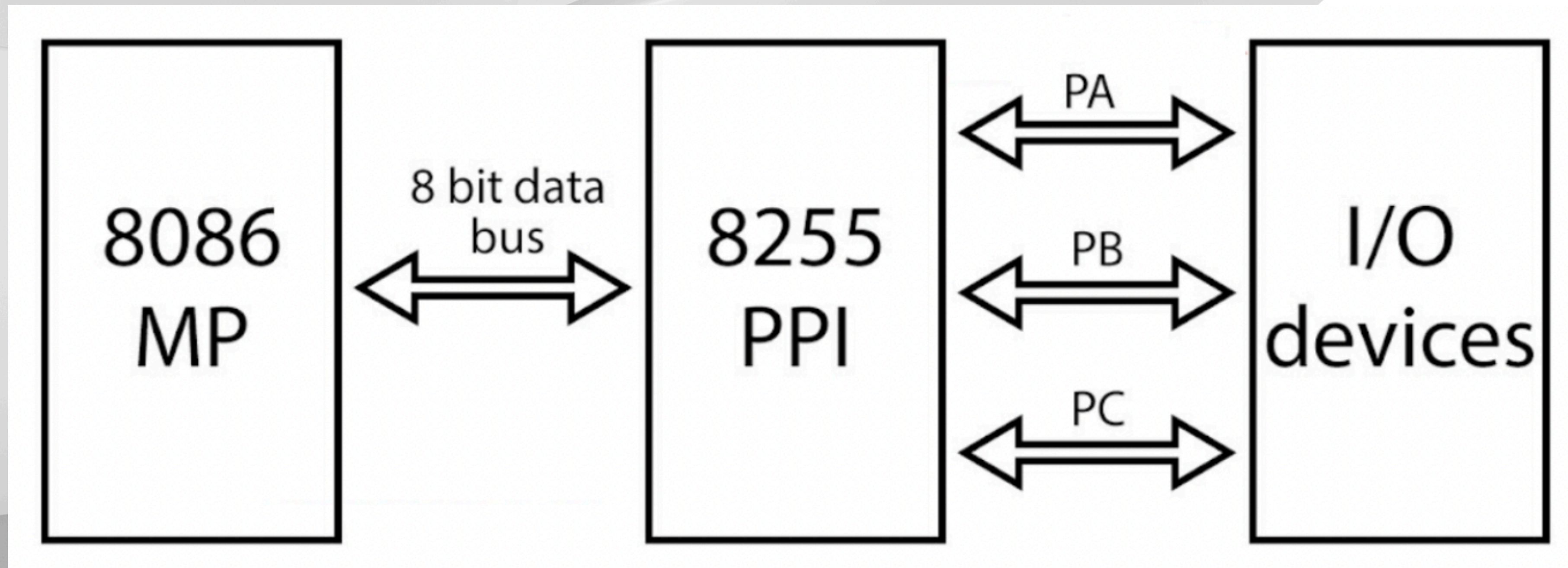
I/O: роль ОС та bare metal

Сам по собі процесор нічого не знає про пристрої введення-виведення. Його інструкції призначені лише для роботи з регістрами та пам'яттю. Коли ми хочемо виконати I/O, наприклад, вивести символ на екран чи прочитати натискання клавіші, програма викликає службу операційної системи. ОС містить драйвери й знає, як взаємодіяти з конкретними пристроями. Але що робити, якщо ОС немає? У випадку bare metal програмування доводиться безпосередньо звертатися до портів, регістрів і контролерів обладнання, використовуючи спеціальні інструкції процесора.

Порти введення-виведення та підключені пристрої

У класичній архітектурі x86 пристрої введення-виведення доступні через спеціальні адреси — порти. Це окремий простір, відмінний від адрес пам'яті. Процесор може читати дані з пристроїв або надсилати їм команди за допомогою інструкцій IN та OUT. Наприклад, можна звернутися до порту клавіатури, щоб отримати код натиснутої клавіші, або до порту відеоадаптера, щоб вивести символ на екран. До портів підключаються різні контролери: клавіатури, миші, відео, звукові карти, мережеві адаптери та інші пристрої.

Порти введення-виведення



Як процесор виконує I/O: інструкції IN та OUT

Для взаємодії з пристроями процесор архітектури x86 використовує спеціальні інструкції IN та OUT. Вони дозволяють читати дані з порту введення-виведення або надсилати дані до нього. Наприклад, `IN AL, 60h` читає байт із порту клавіатури, а `OUT 64h, AL` може передати команду контролеру. Кожен порт має свій унікальний номер (адресу), за яким звертається процесор. Завдяки цим командам програма отримує можливість напряду взаємодіяти з апаратними пристроями без участі операційної системи, що особливо важливо у режимі `bare metal`.

Читання подій з пристроїв через порти

Події від клавіатури чи миші насправді є даними, які процесор зчитує з певних портів введення-виведення. Наприклад, при натисканні клавіші контролер клавіатури записує код у свій порт (звичайно 60h), а програма може отримати цей байт командою IN. Аналогічно, переміщення миші чи натискання її кнопок генерує коди, що зберігаються у відповідних портах. Таким чином, будь-яка взаємодія користувача з пристроями перетворюється на просту операцію читання значення з конкретної адреси порту.

OUT: надсилання даних до пристроїв

Інструкція OUT використовується для передавання даних із процесора до пристроїв через порти введення-виведення. Наприклад, щоб надрукувати символ на принтері, байт відправляється у відповідний порт контролера принтера. Аналогічно, відправка пакета у мережу починається з передачі байтів у порт мережевого адаптера. Навіть простий ефект, як-от вмикання або вимикання світлодіода на клавіатурі, реалізується через OUT — процесор надсилає керуючу команду у порт клавіатурного контролера. Таким чином, OUT є ключем до керування зовнішніми пристроями.

IN: повний синтаксис.

Інструкція	Опис
<code>IN AL, imm8</code>	Зчитати байт із порту imm8 → AL
<code>IN AX, imm8</code>	Зчитати слово із порту imm8 → AX
<code>IN EAX, imm8</code>	Зчитати подвійне слово → EAX
<code>IN AL, DX</code>	Зчитати байт із порту DX → AL
<code>IN AX, DX</code>	Зчитати слово із порту DX → AX
<code>IN EAX, DX</code>	Зчитати подвійне слово → EAX

OUT: повний синтаксис.

Інструкція	Опис
<code>OUT imm8, AL</code>	Записати байт AL → порт imm8
<code>OUT imm8, AX</code>	Записати слово AX → порт imm8
<code>OUT imm8, EAX</code>	Записати подвійне слово EAX
<code>OUT DX, AL</code>	Записати байт AL → порт DX
<code>OUT DX, AX</code>	Записати слово AX → порт DX
<code>OUT DX, EAX</code>	Записати подвійне слово EAX

Нумерація портів і залежність від архітектури

Нумерація портів введення-виведення не є універсальною та жорстко визначається архітектурою процесора й рішеннями виробника обладнання. Кожен пристрій підключається до свого діапазону портів, які закріплені стандартами або домовленостями між розробниками апаратури. Наприклад, контролер клавіатури традиційно використовує порт 60h для даних і 64h для команд, відеоадаптери мають власні діапазони, так само як мережеві чи звукові карти. Тому програміст повинен знати точні адреси портів конкретної платформи.

Виробники і розташування портів

У різних виробників апаратури немає обов'язкової єдиної схеми підключення всіх пристроїв. Хоча для базових компонентів, як-от клавіатура чи системний таймер, існують усталені адреси портів, додаткові контролери можуть бути «припаяні» до інших номерів. Наприклад, один виробник материнських плат може призначити порти для USB-інтерфейсу в класичному діапазоні, а інший — змістити їх у нову область. Аналогічно з мережевими чи звуковими картами: їхні драйвери орієнтовані на ті номери портів, які визначив конкретний виробник обладнання.

Чому прямий доступ до портів майже зник

Прямий доступ до портів введення-виведення сьогодні практично не використовується у звичайному програмуванні.

Основна причина — залежність від конкретної апаратури та відсутність єдиних стандартів для всіх виробників.

Будь-яка зміна у схемі підключення робить такий код нестабільним і непридатним для перенесення.

Натомість драйвери та операційні системи приховують деталі роботи з портами, забезпечуючи єдиний інтерфейс.

Прямий I/O зберігся лише у виробників «заліза» і використовується для низькорівневого програмування BIOS/Firmware.

Апартні переривання: реакція на події

Не всі пристрої працюють за принципом «запитав — отримав». Наприклад, у мережі ми не можемо знати, коли прийде новий байт і скільки їх буде. Постійне опитування портів витрачає ресурси і неефективне. Для цього в апаратурі існує механізм hardware interrupt — коли пристрій сам сигналізує процесору про появу нових даних або подій. Процесор перериває виконання поточного коду та переходить до спеціальної обробної процедури (ISR), яка зчитує дані чи виконує необхідні дії. Це дозволяє ефективно реагувати на асинхронні події.

Interrupt Descriptor Table (IDT)

Щоб процесор знав, як реагувати на різні переривання, використовується спеціальна структура — Interrupt Descriptor Table (IDT). Це таблиця в пам'яті, де для кожного номера переривання зберігається адреса обробника та додаткові атрибути. Коли пристрій генерує сигнал, процесор визначає номер переривання і звертається до IDT, щоб знайти відповідну процедуру. Завдяки цьому система може швидко переключатися між десятками різних джерел подій — від клавіатури і миші до таймера та мережевих адаптерів.

Interrupt Descriptor Table (IDT)

У архітектурі x86 кількість можливих апаратних переривань визначається розміром таблиці IDT:

IDT має 256 записів (номери 0–255).

Частина з них зарезервована:

0–31 — винятки процесора

(ділення на нуль, помилки пам'яті, інструкцій тощо).

32–255 — можуть бути використані

для апаратних і програмних переривань.

Таким чином, апаратних переривань може бути до **224** різних джерел (залежить від чипсета й контролерів).

Залежність переривань від виробника

Хоча процесор підтримує єдину схему роботи з перериваннями через IDT, реальне використання цієї технології суттєво залежить від виробника обладнання. Кожен контролер чи пристрій має власний спосіб сигналізувати події, і саме виробник визначає, який номер переривання буде використано та які додаткові параметри потрібні для його обробки. Тому на різних платформах однакові пристрої можуть мати різні номери IRQ або навіть різні механізми доступу. Це ускладнює пряме програмування «заліза» і робить необхідними драйвери та стандарти.

Роль драйверів у роботі з обладнанням

Через складність і залежність роботи переривань та портів від конкретного виробника, пряме програмування обладнання є ненадійним і схильним до помилок.

Щоб цього уникнути, використовуються драйвери — спеціальні програми, які створюють самі виробники або розробники обладнання. Драйвери приховують від прикладного програміста всі апаратні деталі: номери портів, IRQ, специфічні протоколи обміну.

Завдяки цьому додатки працюють із єдиним зрозумілим інтерфейсом операційної системи, а відповідальність за коректну взаємодію з «залізом» бере на себе драйвер.

Чому драйвери такі важливі

Різні версії одного й того ж обладнання можуть мати відмінності в апаратній реалізації:

змінені номери портів, інші режими роботи контролерів чи навіть нові функції.

Для прикладної програми це критично, адже вона не може передбачити всі варіанти.

Саме тому драйвери є ключовим елементом: вони знають специфіку конкретної моделі пристрою й адаптують роботу системи під неї.

Додатки звертаються лише до уніфікованого API операційної системи, а драйвер «перекладає» ці запити на правильні послідовності команд для конкретної версії обладнання.

Notes

Notes

Notes